Лабораторная работа №1: Loki, Zabbix и Grafana

Цель работы

Развернуть тестовый сервис Nextcloud и организовать для него сбор логов и мониторинг. Передавать логи через Promtail в Loki и и сделать их доступными для анализа в Grafana. Состояние Nextcloud должно проверяться через Zabbix, а данные отображаться на двух дашбордах Grafana.

Структура проекта

- `docker-compose.yml` описывает Nextcloud, Loki, Promtail, Grafana, Zabbix Server, веб-интерфейс Zabbix и PostgreSQL.
- `promtail_config.yml` задаёт параметры чтения логов Nextcloud и адрес Loki внутри сети Docker Compose.
- `zabbix/template.yml` содержит HTTP-проверку `/status.php`, преобразование состояния `maintenance` в `healthy` или `unhealthy` и триггер.

Почему мы выбрали такие конфигурации

docker-compose.yml

Мы разделили стенд на отдельные сервисы, чтобы запускать и проверять Nextcloud, Promtail, Loki, Grafana и Zabbix независимо. Docker Compose помещает сервисы в общую сеть. ЧТобы приложения внутри стенда обращались друг к другу по именам сервисов, например `nextcloud`, `loki` и `zabbix-front`, а не через `localhost`. `localhost` внутри контейнера указывает на сам этот контейнер

Порты опубликованы на компьютере, чтобы мы могли открыть веб-интерфейсы Nextcloud, Zabbix и Grafana из браузера, а при необходимости проверить Loki. Данные Nextcloud и база Zabbix хранятся в именованных томах, поэтому они не пропадают при обычном пересоздании контейнеров

Для Postgres задана проверка готовности `pg_isready`. Zabbix Server и веб-интерфейс запускаются после того, как база данных готова. сделали чтобы уменьшить вероятность ошибок при первом старте, когда контейнеры поднимаются одновременно

Для простоты аутентификации включена анонимная роль Grafana Admin, а для базы Zabbix заданы простые учётные данные

Ход работы

1. Развёртывание сервисов

Мы проверили конфигурацию командой `docker compose config -q`, запустили стенд командой `docker compose up -d` и проверили состояние контейнеров командой `docker compose ps`.

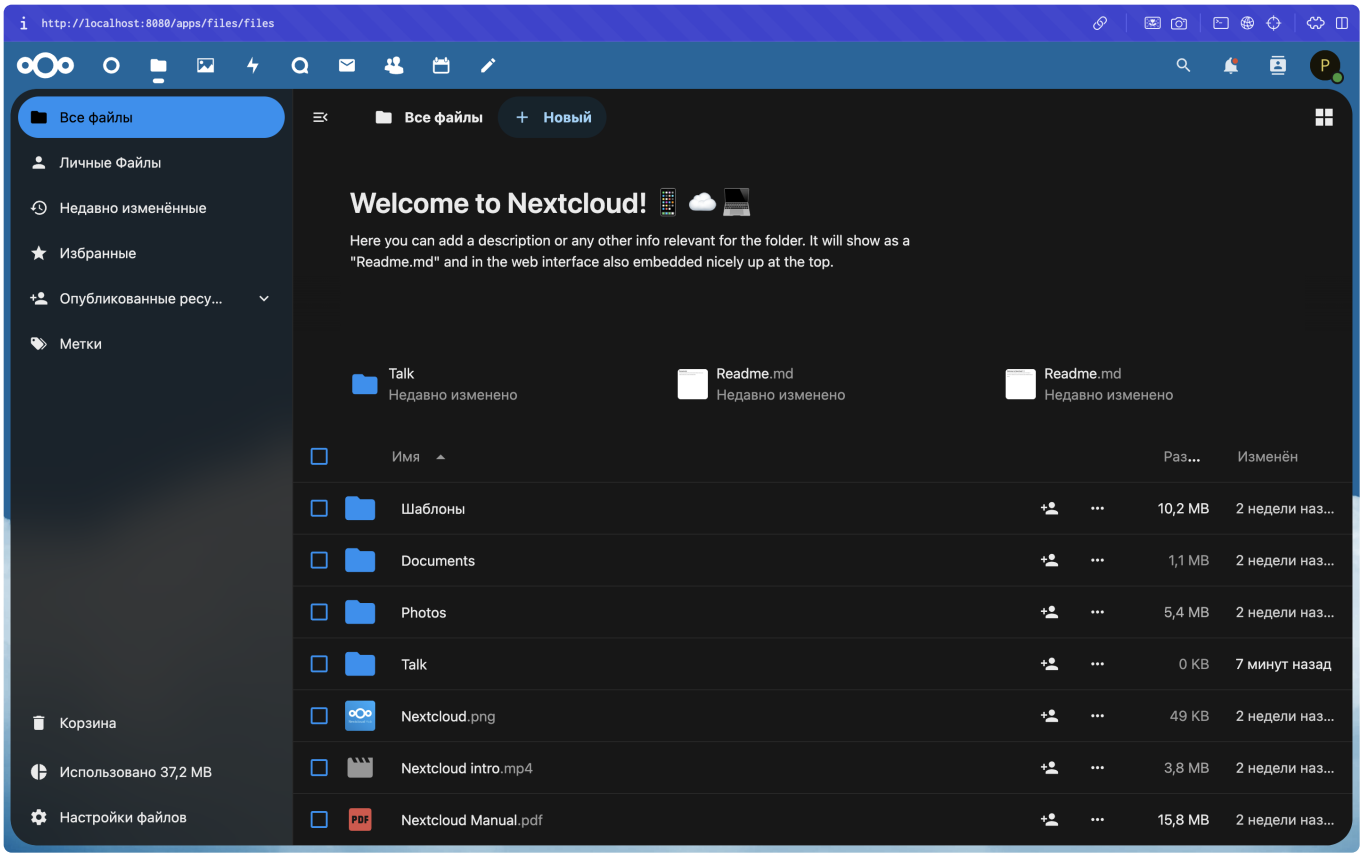
Результат: все семь контейнеров запущены. PostgreSQL и веб-интерфейс Zabbix отмечены Docker как healthy. Конфигурация Compose прошла проверку.

```
└─ docker compose up -d
[+] Running 7/7
 ✔ Container loki              Running                    0.0s
 ✔ Container postgres-zabbix   Healthy                    1.0s
 ✔ Container nextcloud         Running                    0.0s
 ✔ Container zabbix-back       Running                    0.0s
 ✔ Container promtail          Running                    0.0s
 ✔ Container zabbix-front      Running                    0.0s
 ✔ Container grafana           Running                    0.0s
```

2. Инициализация Nextcloud и проверка логирования

Мы открыли `http://localhost:8080`, создали локальную учётную запись Nextcloud и добавили имя контейнера `nextcloud` в список `trusted_domains`. Затем проверили наличие `nextcloud.log` и журналы Promtail. Строка `Seeked /opt/nc_data/nextcloud.log` показывает, что Promtail нашёл файл и начал его читать.

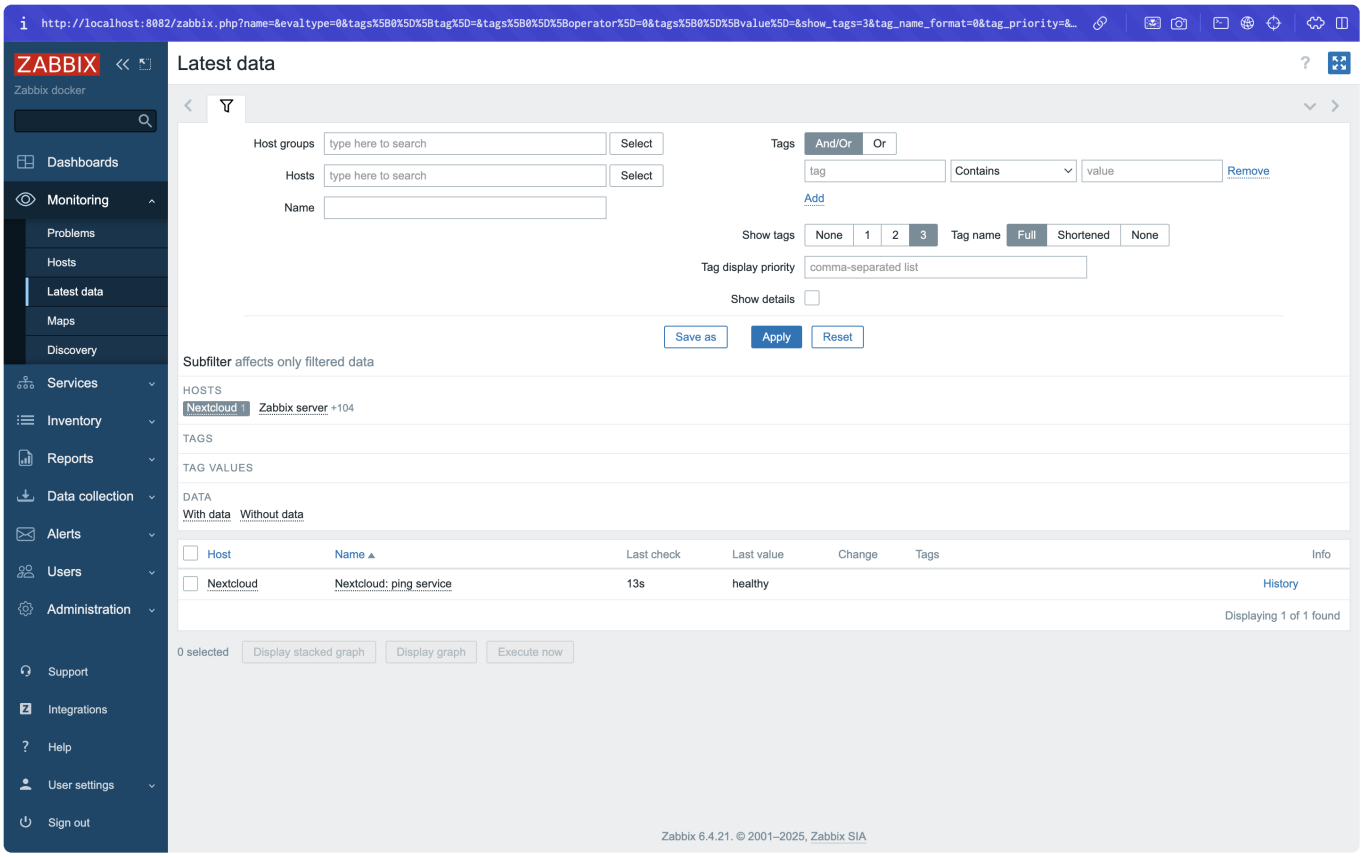
Фактический результат: Nextcloud 29.0.6.1 установлен. В `trusted_domains` настроены `localhost:8080` и `nextcloud`. Файл `/var/www/html/data/nextcloud.log` создан. Promtail читает `/opt/nc_data/nextcloud.log`



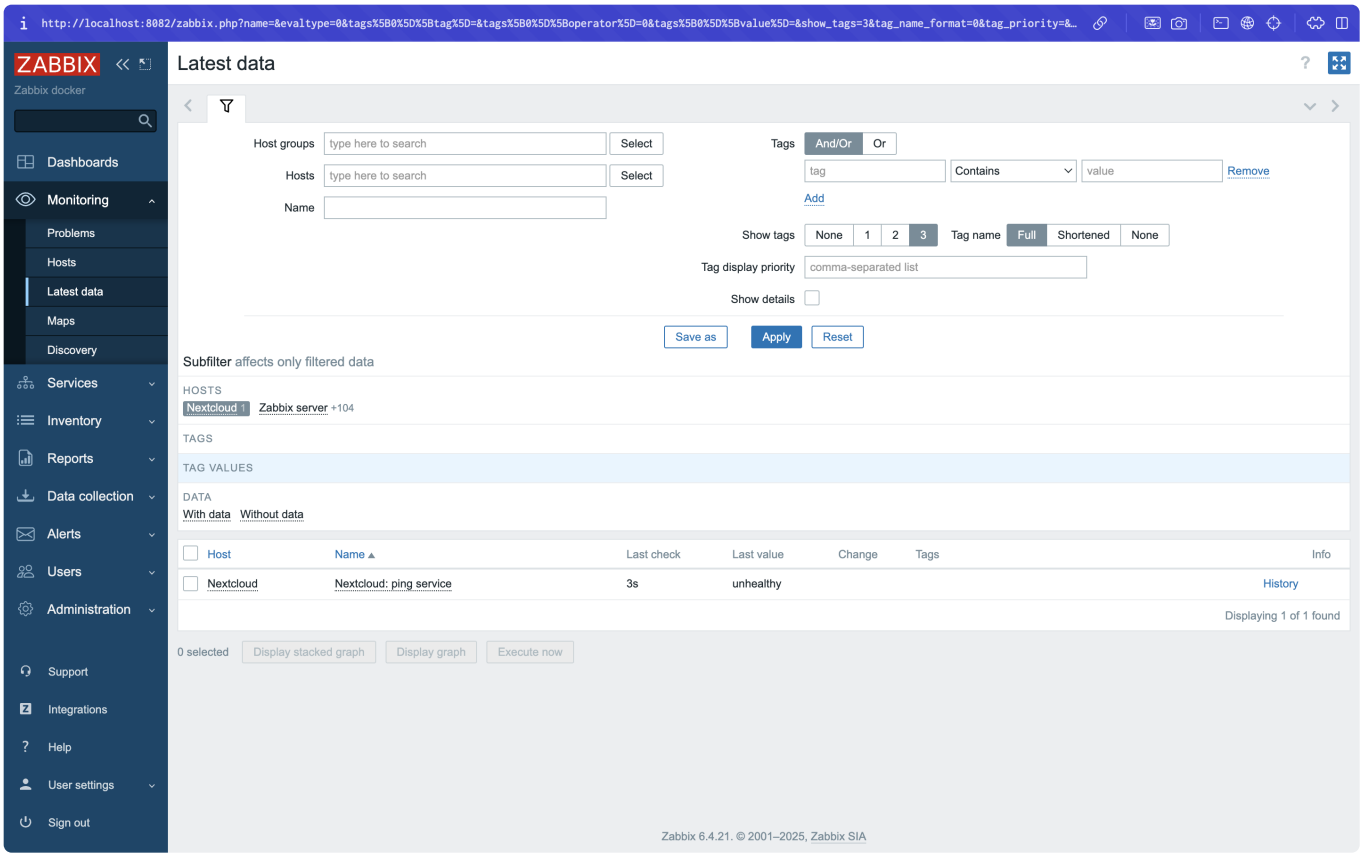
3. Мониторинг в Zabbix

Мы импортировали шаблон `Test ping template`, создали хост с техническим именем `nextcloud` и привязали к нему шаблон. Элемент данных `Nextcloud: ping service` обращается к `http://nextcloud/status.php`

При первой попытке импорта Zabbix сообщил `unexpected tag "triggers"`, тк блок триггера находился на уровне шаблона, что нам не подходит. Мы перенесли блок внутрь в `zabbix/template.yml` и проверили YAML парсером Ruby. Повторный импорт прошёл в Zabbixe появились шаблон `Test ping template`, элемент данных `Nextcloud: ping service` и триггер `Nextcloud is in maintenance mode`. Хост `nextcloud` создан в группе `Applications` и связан с шаблоном



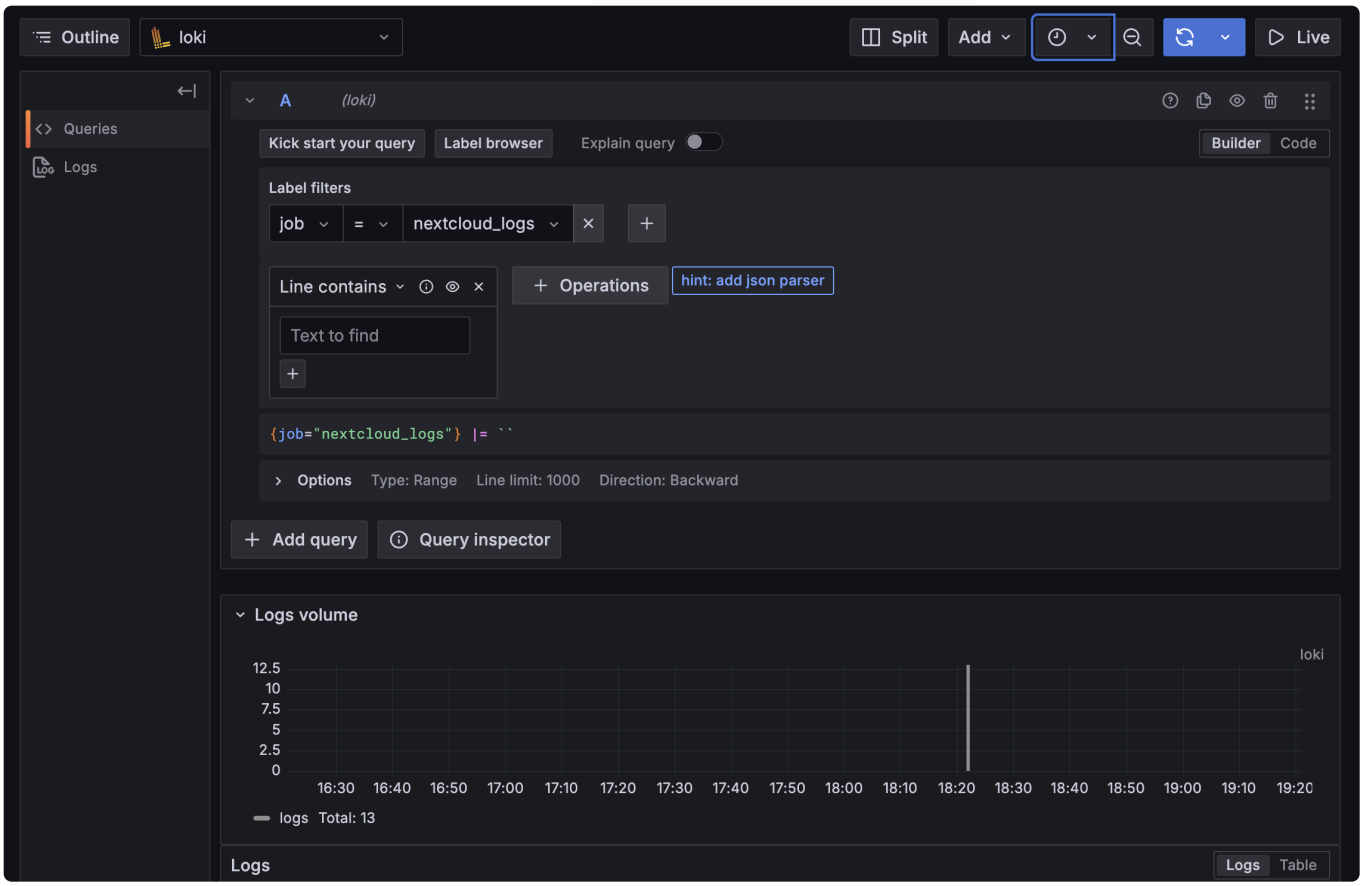
При проверке maintenance mode мы включили режим обслуживания Nextcloud. Хост перешёл в состояние `unhealthy`. После выключения режима обслуживания состояние вернулось к нормальному



4. Источники и дашборды Grafana

Я подключил Loki по адресу `http://loki:3100` и Zabbix API по адресу `http://zabbix-front:8080/api_jsonrpc.php`.

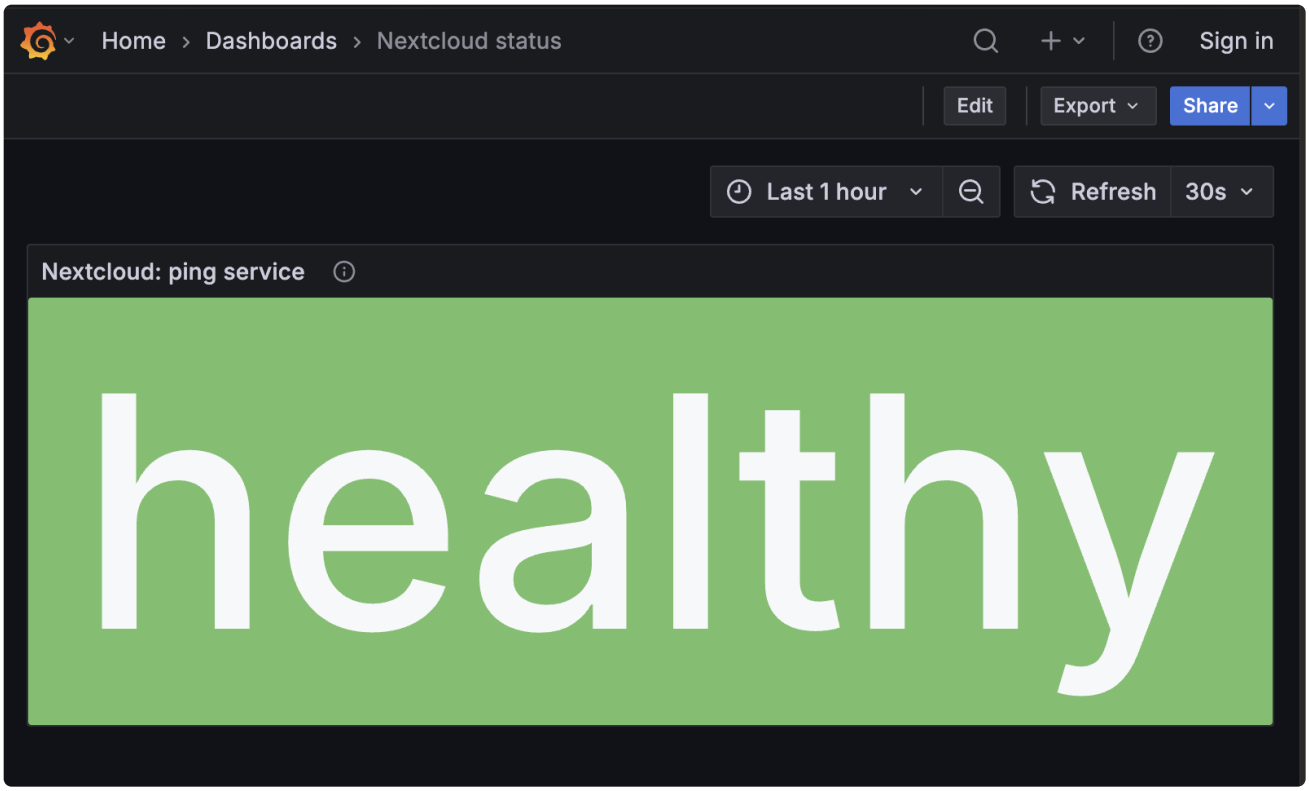
В Explore для Loki использовал запрос `{job="nextcloud_logs"}` и получил:



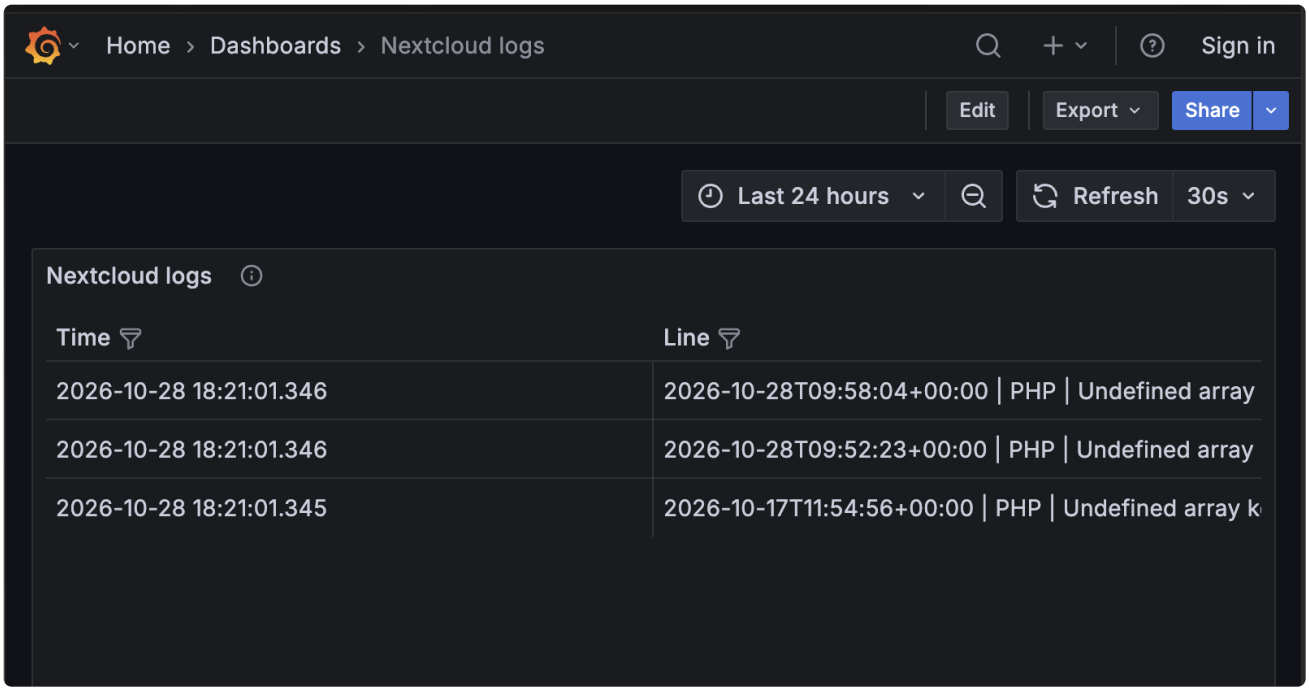
. Для Zabbix использовал элемент `Nextcloud: ping service`.

Созданные дашборды:

1. **Nextcloud status**



2. **Nextcloud logs**



Итог

Развернули тестовый сервис Nextcloud и организовали для него сбор логов и мониторинг. Передали логи через Promtail в Loki и и сделали их доступными для анализа в Grafana. Состояние Nextcloud проверяется через Zabbix, а данные отображаются на двух дашбордах Grafana.

Ответы на вопросы

1. Чем SLO отличается от SLA?

SLO (Service Level Objective) это измеримое значение уровня сервиса, например доступность 99,9% за месяц
SLA (Service Level Agreement) это договорённость с пользователем или заказчиком, которая фиксирует целевые показатели
SLO задаёт техническую цель, а SLA устанавливает соглашения и обязательства между сторонами

2. Чем отличается инкрементальный бэкап от дифференциального?

Инкрементальный бэкап сохраняет изменения с момента последней резервной копии. Для восстановления например нужны последняя полная копия и вся последовательность последующих инкрементальных копий

Дифференциальный бэкап сохраняет изменения с момента последней полной копии. Для восстановления нужны полная копия и последний дифференциальный бэкап.
Инкрементальные копии обычно меньше, а дифференциальные проще для восстанов

3. В чём разница между мониторингом и observability?

Мониторинг проверяет заранее выбранные показатели и уведомляет об известных состояниях или порогах, например упал Nextcloud
Observability или наблюдаемость, это возможность исследовать внутреннее состояние системы по внешним данным: метрикам, логам и трассировкам.
У нас Zabbix проверяет состояние сервиса, а Loki и Grafana для исследования логов и данных